



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پروژه درس معماری کامپیوتر

تحلیل نقصان‌های برخورد و طراحی معماری حافظه نهان قربانی

نگارش

نفیسه زارعی

آرش اکبری

آیناز رحمانی

استاد

دکتر حمید سربازی آزاد

مرداد ۱۴۰۵

چکیده

در این پروژه به بررسی و حل مشکل افت عملکرد ناشی از نقصان‌های برخورد در حافظه‌های نهان با روش نگاشت مستقیم پرداخته شده است. تداخل آدرس‌ها در این ساختار منجر به پدیده تلاطم و مراجعه‌های مکرر و پرهزینه به حافظه اصلی می‌شود. برای رفع این چالش، یک حافظه نهان قربانی تمام‌انجمی طراحی و بین سطح اول حافظه نهان و سطح پایین‌تر قرار داده شد. این معماری در شبیه‌ساز آکیتا پیاده‌سازی شده و عملکرد آن با استفاده از یک برنامه محک با دسترسی‌های متناوب مورد ارزیابی قرار گرفت. نتایج شبیه‌سازی و بررسی شمارنده‌های سخت‌افزاری نشان می‌دهد که افزودن این پیمان به مدیریت صحیح جابه‌جایی دوطرفه، ترافیک حافظه اصلی را به شکل چشمگیری کاهش داده و زمان دسترسی میانگین به حافظه را بهبود می‌بخشد.

کلیدواژه‌ها: حافظه نهان قربانی، نقصان برخورد، نگاشت مستقیم، تلاطم، زمان دسترسی میانگین.

فهرست مطالب

۱	مقدمه	۱
۳	مفاهیم پایه و پیش زمینه	۲
۳	۱-۲ سلسله مراتب حافظه	۳
۳	۲-۲ انواع نقصان در حافظه نهان	۳
۴	۱-۲-۲ نقصان اجباری	۴
۴	۲-۲-۲ نقصان ظرفیتی	۴
۴	۳-۲-۲ نقصان برخورد	۴
۵	۳ ساختار پایه و شبیه سازی وضعیت تلاطم	۵
۵	۱-۳ معماری پایه در شبیه ساز آکیتا	۵
۶	۱-۱-۳ حافظه نهان در شبیه ساز آکیتا	۶
۶	۲-۱-۳ حافظه اصلی در شبیه ساز آکیتا	۶
۷	۳-۱-۳ نحوه اتصال حافظه نهان و حافظه اصلی	۷
۷	۲-۳ برنامه محک و ایجاد تلاطم	۷
۸	۱-۲-۳ آزمون اول	۸
۸	۲-۲-۳ آزمون دوم	۸
۱۰	۴ طراحی و معماری حافظه نهان قربانی	۱۰

۱-۴	ساختار تمام انجمنی و سیاست جایگزینی	۱۰
۲-۴	مکانیزم تبادل دوطرفه و مسیریابی پیام‌ها	۱۱
۵	ارزیابی و تحلیل نتایج شبیه‌سازی	۱۲
۱-۵	مقایسه ترافیک حافظه و تاخیر سامانه	۱۲
۲-۵	تحلیل ریاضی زمان دسترسی میانگین	۱۲
۶	نتیجه‌گیری	۱۳
	مراجع	۱۴

فصل ۱

مقدمه

شکاف فزاینده بین سرعت پردازنده‌ها و حافظه اصلی یکی از چالش‌های اساسی در معماری کامپیوتر است. برای پر کردن این شکاف از سلسله‌مراتب حافظه نهان^۱ استفاده می‌شود. با این حال محدودیت‌هایی که در ساختارهای نگاشت حافظه وجود دارد، چالش‌های جدیدی را به وجود می‌آورد. به عنوان مثال، به دلیل سادگی و سرعت بالا در پیاده‌سازی، استفاده از حافظه‌های نهان با نگاشت مستقیم^۲ یا انجمنی با درجه پایین بسیار رایج است اما این روش باعث بروز نقصان‌های برخورد^۳ می‌شود. در این حالت ممکن است چندین آدرس پرکاربرد دقیقاً به یک خط از حافظه نهان اختصاص یابند. حتی اگر ظرفیت کلی حافظه برای نگهداری داده‌ها کافی باشد، این تداخل باعث می‌شود داده‌ها پیوسته از حافظه اخراج^۴ شوند.

این رفتار ناکارآمد که به عنوان پدیده تلاطم^۵ شناخته می‌شود، سامانه را مجبور می‌کند تا داده‌ها را بارها از سطوح پایین‌تر واکشی کند. این فرایند نه تنها ترافیک بیهوده‌ای در شبکه ارتباطی حافظه ایجاد می‌کند، بلکه تاخیر سنگینی را به پردازنده تحمیل می‌نماید. برای حل این مشکل، ایده اضافه کردن یک بافر بسیار کوچک و تمام‌انجمنی^۶ به عنوان حافظه نهان قربانی^۷ مطرح می‌شود که داده‌های اخراجی از سطح اول را پیش از انتقال به سطح پایین‌تر در خود نگه می‌دارد.

در این پروژه روند ناکارآمد ذکر شده ابتدا در قالب یک معماری پایه شبیه‌سازی و مدل‌سازی شده است تا میزان هدررفت عملکرد سامانه اندازه‌گیری شود. سپس با طراحی یک پیمانۀ جدید و تغییر منطق مسیریابی

Cache^۱

Direct-Mapped^۲

Conflict Misses^۳

Evict^۴

Thrashing^۵

Fully Associative^۶

Victim Cache^۷

پیام‌ها^۸، سامانه بهینه شده است. هدف نهایی این گزارش تحلیل تاثیر این معماری جدید بر موازنه میان هزینه ساخت‌افزایی و زمان دسترسی میانگین^۹ برنامه‌ها و همچنین بررسی روند جلوگیری از بن‌بست^{۱۰} در تبادل هم‌زمان پیام‌ها است.

فصل ۲

مفاهیم پایه و پیش زمینه

در این فصل مفاهیم پایه‌ای مرتبط با ساختار حافظه نهان^۱ و چالش‌های عملکردی آن بررسی می‌شود. این مفاهیم برای درک بهتر اهداف این پروژه از جمله تشخیص انواع نقصان‌ها و طراحی معماری حافظه نهان قربانی^۲ ضروری هستند.

۱-۲ سلسله مراتب حافظه

در معماری‌های پایه و مرسوم، زمانی که پردازنده به داده‌ای نیاز دارد ابتدا آن را در حافظه نهان سطح اول جستجو می‌کند. اگر داده به دلیل نقصان در این سطح یافت نشود، از سطوح پایین‌تر واکشی می‌شود. هنگام ورود یک بلاک جدید به حافظه نهان سطح اول، ممکن است بلاک قدیمی موجود در آن جایگاه بلافاصله اخراج^۳ شده و به سطوح پایین‌تر فرستاده شود. در صورتی که پردازنده دوباره به همان بلاک اخراج شده نیاز پیدا کند، سامانه باید تاخیر سنگین خواندن دوباره آن از سطوح پایین‌تر را تحمل کند.

۲-۲ انواع نقصان در حافظه نهان

هنگام دسترسی پردازنده به حافظه، در صورتی که داده مورد نظر در سطح فعلی موجود نباشد، پدیده‌ای به نام نقصان اتفاق می‌افتد. نقصان‌ها به طور کلی به سه دسته اصلی تقسیم می‌شوند که درک تفاوت میان آن‌ها برای تحلیل عملکرد سیستم اهمیت بالایی دارد.

^۱ Cache Memory

^۲ Victim Cache

^۳ Evict

۱-۲-۲ نقصان اجباری

نقصان اجباری^۴ زمانی رخ می‌دهد که پردازنده برای اولین بار به یک آدرس از حافظه دسترسی پیدا می‌کند. از آنجا که داده مربوط به این آدرس پیش از این خوانده نشده است، وقوع این نقصان و واکنشی آن از سطوح پایین‌تر اجتناب‌ناپذیر است.

۲-۲-۲ نقصان ظرفیتی

نقصان ظرفیتی^۵ زمانی اتفاق می‌افتد که گنجایش کلی حافظه نهان برای نگهداری تمام داده‌های مورد نیاز برنامه کافی نباشد. در پیاده‌سازی این پروژه و در بخش ایجاد کننده درخواست‌ها، این پدیده با واکنشی آدرس‌هایی خارج از گستره ظرفیت حافظه نهان به صورت عملی مدل‌سازی شده است.

۳-۲-۲ نقصان برخورد

نقصان برخورد^۶ به طور ویژه در حافظه‌های نهانی رخ می‌دهد که با روش نگاشت مستقیم^۷ یا انجمنی^۸ با درجه پایین کار می‌کنند. به دلیل محدودیت‌هایی که در ساختارهای نگاشت حافظه وجود دارد، ممکن است چندین آدرس پرکاربرد به یک خط مشخص از حافظه نهان اختصاص یابند. حتی اگر ظرفیت کلی حافظه نهان برای نگهداری داده‌ها کافی باشد، تداخل آدرس‌ها باعث می‌شود داده‌ها پیوسته از حافظه اخراج شده و پدیده‌ای به نام تلاطم^۹ رخ دهد. این اتفاق سامانه را مجبور می‌کند تا داده‌ها را بارها از سطوح پایین‌تر حافظه واکنشی کند که جریمه زمانی و تاخیر قابل توجهی را به همراه دارد. در کدهای شبیه‌سازی این پروژه، این رفتار ناکارآمد با ایجاد حلقه‌ای از درخواست‌های مکرر بر روی آدرس‌های متعارض پیاده‌سازی شده است.

Compulsory Miss^۴

Capacity Miss^۵

Conflict Miss^۶

Direct-Mapped^۷

Associative^۸

Thrashing^۹

فصل ۳

ساختار پایه و شبیه‌سازی وضعیت تلاطم

در این فصل به بررسی ساختار پایه شبیه‌ساز و نحوه مدل‌سازی رفتار پردازنده برای ایجاد ترافیک حافظه می‌پردازیم. پیش از اضافه کردن هرگونه راهکار بهبود عملکرد، نیاز است تا معماری پایه به درستی مستقر شده و پدیده تلاطم^۱ که منجر به افت شدید عملکرد سیستم می‌شود به صورت دقیق شبیه‌سازی و اندازه‌گیری شود. برای این منظور از تولید الگوهای دسترسی متناوب به حافظه استفاده شده است.

۳-۱ معماری پایه در شبیه‌ساز آکیتا

شبیه‌ساز آکیتا^۲ یک چارچوب شبیه‌سازی رویدادمحور^۳ در معماری کامپیوتر است که به زبان گو^۴ توسعه یافته است. این شبیه‌ساز از قطعات سخت‌افزاری مستقلی تشکیل شده است که هر کدام دارای پورت‌های ارتباطی مشخصی هستند و با ارسال پیام با یکدیگر ارتباط برقرار می‌کنند. آکیتا به طراحان اجازه می‌دهد تا زمان‌بندی دقیق چرخه‌های ساعت، تاخیر در صف‌های ارسال پیام و عملکرد قطعات حافظه را با دقت بالایی مدل‌سازی کنند. در این پروژه، از آکیتا برای ساخت یک سیستم پایه متشکل از یک پردازنده تولیدکننده الگو، یک سطح از حافظه نهان و یک کنترل‌کننده حافظه اصلی استفاده شده است.

^۱Thrashing

^۲Akita

^۳Event-driven

^۴Go

۳-۱-۱ حافظه نهان در شبیه‌ساز آکیتا

پیکربندی حافظه نهان سطح یک (L1) در معماری پایه در فایل main.go و با استفاده از سازنده writeback پیاده‌سازی شده است. مشخصات و ویژگی‌های عملکردی این حافظه به شرح زیر است:

- **ظرفیت و اندازه بلاک:** ظرفیت کل این حافظه برابر یک کیلوبایت (1 KB) در نظر گرفته شده است. با توجه به تنظیم مقدار لگاریتم اندازه بلاک برابر ۶ ($\text{Log2BlockSize} = 6$)، اندازه هر بلاک داده در این حافظه ۶۴ بایت است. این یعنی حافظه نهان در مجموع دارای ۱۶ بلاک فیزیکی است.
 - **سیاست نگاشت:** درجه انجمنی^۵ این حافظه برابر ۱ تنظیم شده است. این مقدار بیانگر استفاده از روش نگاشت مستقیم^۶ است. در این روش، هر آدرس حافظه دقیقاً به یک خط مشخص در حافظه نهان نگاشت می‌شود که زمینه را برای بروز نقصان‌های برخورد^۷ فراهم می‌کند.
 - **تاخیر دسترسی:** تاخیر اصابت^۸ برابر ۱ چرخه ساعت است که به صورت مساوی (نیم چرخه) بین تاخیر دایرکتوری و تاخیر بانک‌های حافظه تقسیم شده است.
 - **مدیریت درخواست‌های معلق:** این ماژول دارای ۴ مدخل در ثبات نگهداری وضعیت نقصان^۹ است. این ویژگی به حافظه نهان اجازه می‌دهد در صورت وقوع نقصان، تا ۴ درخواست را به صورت هم‌زمان به سطوح پایین‌تر ارسال کرده و منتظر پاسخ بماند بدون این که متوقف شود.
- علاوه بر موارد فوق، این ماژول از سیاست بازنویسی^{۱۰} استفاده می‌کند، به این معنا که تغییرات داده تا زمانی که خط مورد نظر از حافظه اخراج نشود، به حافظه اصلی فرستاده نخواهد شد.

۳-۱-۲ حافظه اصلی در شبیه‌ساز آکیتا

برای مدل‌سازی حافظه اصلی در حالت پایه، از یک کنترل‌کننده حافظه ایده‌آل^{۱۱} استفاده شده است. این کنترل‌کننده پیچیدگی‌های داخلی ماژول‌های رم^{۱۲} (مانند تازه کردن ردیف‌ها و تاخیرهای متغیر بانک‌ها) را نادیده گرفته و به جای آن یک مدل ساده و قابل پیش‌بینی ارائه می‌دهد. ظرفیت این حافظه ۶۴ کیلوبایت

^۵ Associativity

^۶ Direct-Mapped

^۷ Conflict Miss

^۸ Hit Time

^۹ Miss Status Holding Register - MSHR

^{۱۰} Write-back

^{۱۱} Ideal Memory Controller

^{۱۲} RAM

تعیین شده است که برای برنامه‌های محک^{۱۳} این پروژه کاملاً پاسخگو است. مهم‌ترین مشخصه این بخش، تاخیر بسیار بالای آن است که برابر ۱۰۰ چرخه ساعت ($MEM_LATENCY = 100$) تنظیم شده است. این تاخیر سنگین باعث می‌شود که هر بار داده‌ای در حافظه نهان یافت نشود، پردازنده برای دریافت آن جریمه زمانی بسیار بزرگی را متحمل شود.

۳-۱-۳ نحوه اتصال حافظه نهان و حافظه اصلی

در شبیه‌ساز آکیتا، قطعات سخت‌افزاری از طریق سیستم اتصال مستقیم^{۱۴} به یکدیگر متصل می‌شوند. هر ماژول دارای پورت‌های بالا^{۱۵} (برای دریافت درخواست از قطعات نزدیک‌تر به پردازنده) و پایین^{۱۶} (برای ارسال درخواست به سطوح دورتر حافظه) است. در معماری پایه، پورت حافظه (Mem) در پردازنده به پورت بالای حافظه نهان متصل می‌شود. سپس پورت پایین حافظه نهان مستقیماً به پورت بالای کنترل‌کننده حافظه اصلی وصل می‌گردد. همچنین برای هدایت صحیح بسته‌های داده، از یک نگاشت‌گر پورت یگانه^{۱۷} استفاده شده است تا تمامی درخواست‌های خروجی از حافظه نهان را بدون نیاز به مسیریابی پیچیده، مستقیماً به سمت حافظه اصلی هدایت کند. تمامی این اتصالات در فرکانس ۱ گیگاهرتز همگام‌سازی شده‌اند.

۳-۲ برنامه محک و ایجاد تلاطم

برای سنجش عملکرد حافظه نهان و ارزیابی راه‌حل‌های پیشنهادی، نیاز به تولید ترافیک حافظه داریم. ماژول TraceCPU در این پروژه وظیفه دارد به جای اجرای یک کد اسمبلی واقعی، مستقیماً درخواست‌های خواندن و نوشتن را به آدرس‌های از پیش تعیین شده ارسال کند. در این بخش دو الگوی متفاوت دسترسی به حافظه به نام‌های الگوی کوچک (small trace) و الگوی بزرگ (large trace) طراحی شده است که هدف اصلی آن‌ها ایجاد ترافیک متناوب و شبیه‌سازی بدترین حالت برای حافظه نهان نگاشت مستقیم، یعنی پدیده تلاطم است.

^{۱۳} Benchmark

^{۱۴} Direct Connection

^{۱۵} Top Port

^{۱۶} Bottom Port

^{۱۷} Single Port Mapper

۱-۲-۳ آزمون اول

آزمون اول با استفاده از الگوی smallTrace اجرا می‌شود و هدف آن بررسی رفتار حافظه با تعداد کمی درخواست، اما با نرخ برخورد بالا است. این الگو به سه فاز اصلی تقسیم می‌شود:

۱. فاز پر کردن حافظه: در ابتدا ۱۶ درخواست نوشتن متوالی با فاصله ۶۴ بایت (از آدرس 0x0000 تا 0x03C0) به حافظه ارسال می‌شود. چون ظرفیت حافظه نهان دقیقا ۱۶ بلاک ۶۴ بایتی است، این فاز کل ظرفیت حافظه نهان را پر می‌کند و به طور طبیعی ۱۶ نقصان اولیه^{۱۸} ایجاد می‌کند.

۲. فاز تایید جایگذاری: سپس پردازنده ۶ بلاک اول را دوباره می‌خواند که تمامی این خواندن‌ها منجر به اصابت موفقیت‌آمیز در حافظه نهان می‌شوند.

۳. فاز ایجاد تلاطم: در این مرحله پردازنده درخواست نوشتن در آدرس 0x0400 را ارسال می‌کند. فاصله آدرس 0x0400 (معادل ۱۰۲۴ بایت) با آدرس 0x0000 دقیقا برابر با ظرفیت کل حافظه نهان (۱ کیلوبایت) است. به دلیل ساختار نگاشت مستقیم، هر دو آدرس به یک خط فیزیکی در حافظه نهان ختم می‌شوند. با واکنشی 0x0400، داده 0x0000 اخراج می‌شود. بلافاصله در دستور بعدی، پردازنده دوباره آدرس 0x0000 را درخواست می‌کند که باعث اخراج 0x0400 می‌گردد. این فرایند که داده‌ها پیش از استفاده مجدد از حافظه اخراج می‌شوند، پدیده تلاطم نام دارد. این الگو برای بلاک‌های بعدی نیز تکرار می‌شود تا ترافیک بهبود یافته‌ی روانه حافظه اصلی گردد.

۲-۲-۳ آزمون دوم

آزمون دوم که با متغیر largeTrace شناخته می‌شود، برای شبیه‌سازی یک بار کاری سنگین و طولانی‌تر طراحی شده است تا تاثیر تاخیر حافظه بر زمان کل اجرا را به شکل واضح‌تری نشان دهد. در این آزمون، پردازنده پس از پر کردن اولیه حافظه، وارد یک حلقه تکرار ۳۰ تایی می‌شود. درون این حلقه اصلی، حلقه‌های داخلی دیگری وجود دارند که ۱۰ بار به طور متناوب بین آدرس‌های کاملا متداخل درخواست ارسال می‌کنند (برای مثال تناوب مداوم بین آدرس‌های 0x0000 و 0x0400، و سپس 0x0440 و 0x0040). این حجم از درخواست‌های متناوب به آدرس‌هایی که با یکدیگر برخورد دارند، باعث می‌شود نرخ اصابت در حافظه نهان سطح یک به شدت سقوط کند و سیستم در یک چرخه باطل از اخراج و واکنشی مجدد گرفتار شود. این آزمون بیش از ۶۰۰ دسترسی به حافظه را شبیه‌سازی می‌کند و در معماری پایه، تقریبا تمام این درخواست‌ها به عنوان نقصان برخورد شناخته شده و با تحمیل تاخیر ۱۰۰ چرخه‌ای، باعث کندی

^{۱۸} Cold Miss

شدید زمان نهایی اجرای برنامه می شوند.

فصل ۴

طراحی و معماری حافظه نهان قربانی

در این فصل به بررسی جزئیات طراحی، ساختار سخت‌افزاری و منطق پیاده‌سازی حافظه نهان قربانی^۱ می‌پردازیم. همان‌طور که پیش‌تر اشاره شد، حافظه نهان قربانی یک بافر کوچک و تمام‌انجمنی^۲ است که بین حافظه نهان سطح یک و حافظه اصلی قرار می‌گیرد تا از افت عملکرد ناشی از نقصان‌های برخورد جلوگیری کند^۳. این پیمان در فایل victimcache.go پیاده‌سازی شده است که در ادامه ابعاد مختلف آن تشریح می‌شود.

۴-۱ ویژگی‌های سخت‌افزاری، توپولوژی و ماشین حالت حافظه نهان قربانی

پیکربندی و مشخصات فنی حافظه نهان قربانی در ساختار VCSpec و تابع DefaultVCSpec تعیین شده است. ویژگی‌های کلیدی این ساختار شامل فرکانس کاری ۱ گیگاهرتز، اندازه بلاک ۶۴ بایت (مطبق با اندازه بلاک در حافظه نهان سطح یک) و تعداد ۸ بلاک فیزیکی برای ذخیره‌سازی داده‌های اخراجی است. تاخیر داخلی این ماژول برابر ۵ چرخه ساعت^۴ تنظیم شده است که جریمه بررسی بافر قربانی را مدل‌سازی می‌کند.

^۱ Victim Cache
^۲ Fully Associative
^۳ Conflict Misses
^۴ Latency = 5

۴-۱-۱ توپولوژی و نحوه اتصال پورت‌ها

در معماری دارای حافظه نهان قربانی، نحوه اتصال قطعات تغییر می‌کند تا ترافیک خروجی از حافظه نهان سطح یک مستقیماً به حافظه اصلی نرود. این توپولوژی در فایل main.go به کمک سیستم اتصال مستقیم پیاده‌سازی شده است:

- **پورت بالایی (TopPort):** این پورت به عنوان ورودی حافظه قربانی عمل کرده و به پورت پایینی حافظه نهان سطح یک متصل می‌شود. تمامی درخواست‌های خواندن و نوشتن که از سطح یک خارج می‌شوند (مانند نقصان‌ها و اخراج‌ها) به این پورت وارد می‌شوند.
- **پورت پایینی (BottomPort):** این پورت به عنوان خروجی حافظه قربانی به شمار می‌رود و مستقیماً به پورت بالای کنترل‌کننده حافظه اصلی متصل می‌گردد. درخواست‌هایی که در بافر قربانی نیز یافت نشوند (نقصان در قربانی) یا بلاک‌هایی که به دلیل تکمیل ظرفیت باید اخراج شوند، از طریق این پورت به حافظه اصلی ارسال می‌گردند.

۴-۱-۲ ماشین حالت و مدیریت تراکنش‌ها

ماژول حافظه نهان قربانی به عنوان یک کامپوننت زمان‌دار^۵ در هر چرخه ساعت تابع Tick را اجرا می‌کند. مدیریت تراکنش‌ها در این ماژول از طریق یک ماشین حالت ناهمگام انجام می‌شود که چرخه حیات هر درخواست را کنترل می‌کند:

۱. **دریافت درخواست:** پیام‌های ورودی از طریق TopPort دریافت شده و در صف تراکنش‌های بالا^۶ قرار می‌گیرند.
۲. **مدیریت تاخیر:** تابع countDown در هر چرخه ساعت، مقدار شمارنده تاخیر باقی‌مانده^۷ را برای تراکنش‌های در صف کاهش می‌دهد تا تاخیر سخت‌افزاری ۵ چرخه‌ای اعمال شود.
۳. **پردازش سرصف:** پس از اتمام تاخیر، تابع processHeadTransaction تراکنش را بر اساس نوع آن (خواندن یا نوشتن) به توابع تخصصی هدایت می‌کند تا اصابت یا نقصان بررسی شود.

^۵ Ticking Component
^۶ vcTopTransaction
^۷ CycleLeft

۲-۴ مکانیزم جابه‌جایی دوطرفه، سیاست جایگزینی و مدیریت ناهمگام

منطق اصلی حافظه نهان قربانی در فایل victimcache.go و از طریق توابع پردازش خواندن، نوشتن و پاسخ‌های حافظه پیاده‌سازی شده است که در ادامه به تفکیک بررسی می‌شوند.

۱-۲-۴ ساختار بافر تمام‌انجمی

برخلاف حافظه نهان سطح یک که از نگاشت مستقیم استفاده می‌کند، حافظه نهان قربانی به صورت یک آرایه از ساختارهای VCBLOCK پیاده‌سازی شده است که هر مدخل شامل برچسب^۸، بیت اعتبار^۹، آرایه داده^{۱۰} و زمان آخرین دسترسی است. به دلیل عدم استفاده از تابع نگاشت ریاضی، هنگام جستجوی یک آدرس، کد با استفاده از یک حلقه تمام خانه‌های آرایه را بررسی می‌کند تا اصابت یا نقصان را تشخیص دهد که این دقیقاً رفتار سخت‌افزاری یک بافر تمام‌انجمی را بازسازی می‌کند.

۲-۲-۴ مکانیزم جابه‌جایی دوطرفه و خاصیت انحصاری

هنگامی که پردازنده در حافظه نهان سطح یک دچار نقصان می‌شود اما داده مورد نظر در حافظه نهان قربانی موجود است (اصابت در قربانی)، فرایند جابه‌جایی دوطرفه^{۱۱} رخ می‌دهد. این فرایند در تابع handleRead پیاده‌سازی شده است: داده درخواستی به سرعت به سمت حافظه نهان سطح یک بازگردانده می‌شود. برای حفظ خاصیت انحصاری^{۱۲} و جلوگیری از وجود نسخه تکراری از داده در سطح یک و قربانی، بلافاصله پس از اصابت، بیت اعتبار آن مدخل در قربانی غیرمعتبر می‌گردد (Valid = false). این کار باعث می‌شود جایگاه مورد نظر در قربانی آزاد شود تا بلاک اخراجی بعدی بتواند در آن جای گیرد.

۳-۲-۴ سیاست جایگزینی کمترین استفاده اخیر

هنگامی که حافظه نهان سطح یک خطی را اخراج می‌کند و آن را به حافظه قربانی می‌فرستد (handleWrite)، اگر بافر قربانی کاملاً پر باشد، نیاز به یک سیاست جایگزینی است. در این کد از الگوریتم کمترین استفاده

^۸Tag

^۹Valid

^{۱۰}Data

^{۱۱}Swap

^{۱۲}Exclusive Property

اخیر^{۱۳} استفاده شده است: کد با بررسی فیلد LastAccessTime در تمامی بلاک‌های معتبر، قدیمی‌ترین بلاک را شناسایی می‌کند. سپس آن بلاک قدیمی را از طریق BottomPort به عنوان یک درخواست نوشتن به حافظه اصلی منتقل می‌کند تا داده‌های تغییر یافته از دست نروند. سپس داده جدید جایگزین آن بلاک می‌شود و زمان دسترسی آن به روز می‌گردد.

۴-۲-۴ مدیریت پیام‌های ناهمگام و جلوگیری از بن‌بست

به دلیل تاخیر بالای حافظه اصلی، سیستم نباید متوقف شود. مدیریت ناهمگام پاسخ‌ها و جلوگیری از بروز بن‌بست^{۱۴} از طریق صف vcPendingBottom انجام می‌شود: وقتی درخواستی در قربانی یافت نمی‌شود (نقصان در قربانی)، یا وقتی یک بلاک به ناچار به سمت حافظه اصلی اخراج می‌شود، درخواست مربوطه همراه با شناسه‌های اصلی آن در صف pendingBottom ذخیره می‌گردد و کنترل به شبیه‌ساز بازمی‌گردد. هنگامی که حافظه اصلی پس از گذشت چرخه‌های فراوان پاسخ خود را برمی‌گرداند، توابع completeBottomRead و completeBottomWrite پیام برگشتی را دریافت کرده، با استفاده از شناسه‌ها مرجع اولیه آن را پیدا می‌کنند و پاسخ را به مقصد درست در بالا هدایت می‌نمایند. در مورد اخراج‌های پس‌زمینه^{۱۵} نیز با تنظیم پرچم مربوطه مشخص می‌شود که نیازی به ارسال پاسخ به سمت بالا نیست، که این رویکرد از ایجاد حالت‌های معلق و بن‌بست در شبکه جلوگیری می‌کند.

^{۱۳} Least Recently Used - LRU

^{۱۴} Deadlock

^{۱۵} Eviction

فصل ۵

ارزیابی و تحلیل نتایج شبیه‌سازی

میو

۵-۱ مقایسه ترافیک حافظه و تاخیر سامانه

میو

۵-۲ تحلیل ریاضی زمان دسترسی میانگین

میو

فصل ۶

نتیجه گیری

میور

References